

Module 1: Lexical Analysis

Language Processors, Buffering, Thompson NFA & Direct DFA

Module I: Lexical Analysis — Complete 7-Topic Syllabus Tracker

- Topic 1:** Introduction to Compilers & Its Cousins (P-I-C-A-L)
- Topic 2:** Structure of a Compiler (Phases, Symbol Table, Errors)
- Topic 3:** Lexical Analyzer (Token, Lexeme, Pattern Triad)
- Topic 4:** Input Buffering (Two-Buffer Scheme, Sentinels)
- Topic 5:** Specification of Tokens (Regular Expressions & Operators)
- Topic 6:** Recognition of Tokens (Transition Diagrams & DFA)
- Topic 7:** DFA Directly from Regular Expressions (FLF Rules)
- Exam Prep:** Active Recall Check & Exam-Important Questions

Topic 1: Introduction to Compilers and its Cousins

1. What is a Compiler?

A **compiler** is a specialized software program that translates a source program written in a high-level language into an equivalent target program, usually in machine code or assembly language.

High-Level Language (C, C++, Java) → [COMPILER] → Target Machine / Assembly Code

Parameter	Compiler	Interpreter
Translation Mode	Translates the entire source program into target machine code before execution.	Translates and executes the source program statement-by-statement at runtime.
Execution Speed	Very fast target execution once compiled (standalone binary).	Slower execution due to continuous interpretation overhead.
Memory Requirement	Generates intermediate object files; requires memory for full compilation.	Minimal initial memory; does not produce a standalone object binary.
Error Reporting	Reports all syntactic & semantic errors after full program scan.	Halts execution at the very first line containing an error.

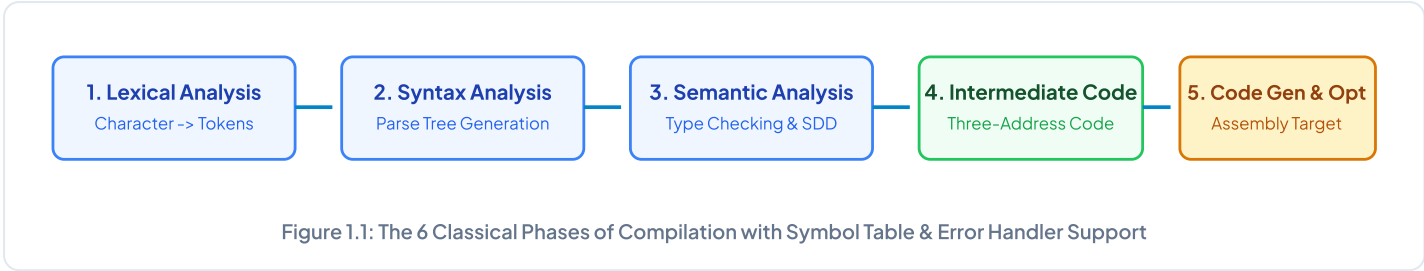
2. Cousins of the Compiler (Language Processing Ecosystem):

- 1. Preprocessor:** Handles source-level directives before compilation (e.g., `#include`, `#define` macro expansion, file inclusion, conditional compilation `#ifdef`).
- 2. Assembler:** Translates human-readable assembly mnemonics (e.g., `MOV R1, R2`) into relocatable binary machine code.
- 3. Linker:** Resolves external cross-file symbol references and links external precompiled library binaries into a single executable module.
- 4. Loader:** Dynamically loads the executable binary into physical RAM, allocates stack/heap segments, adjusts relocatable memory addresses, and initializes the CPU Program Counter (PC).

Memory Hook: P-I-C-A-L

Preprocessor → Compiler → Assembler → Linker → Loader

Topic 2: Structure of a Compiler (Front End vs. Back End)



Cross-Phase Supporting Components:

Symbol Table Management: A centralized data structure (e.g., chained hash table) storing variable and function identifiers, data types, memory offsets, scope levels, and register assignments.

Error Detection & Handling: Detects lexical errors (invalid characters), syntax errors (mismatched parentheses), and semantic errors (type mismatches) and provides recovery mechanisms to continue parsing without crashing.

Topic 3: Lexical Analyzer (Tokens, Lexemes, Patterns)

Concept	Definition	Concrete Example
Token	An abstract syntactic category/class passed to the parser.	`IDENTIFIER`, `NUMBER`, `ASSIGN_OP`, `KEYWORD`
Lexeme	The actual concrete sequence of characters in the source code matching a pattern.	`count`, `100`, `=`, `while`
Pattern	The formal grammar/regular expression rule describing valid lexemes.	`letter(letter digit)*`, `[0-9]+`

Example: Lexical Tokenization Breakdown for `count = 25;`

Lexeme	Token Name	Attribute Value (Symbol Table Pointer / Numeric Constant)
`count`	`IDENTIFIER`	Pointer to Symbol Table entry for `count`
`=`	`ASSIGN_OP`	None
`25`	`NUMBER`	Integer value `25`
`;`	`SEMICOLON`	None

Topic 4: Input Buffering (Two-Buffer Scheme & Sentinels)

Reading one character at a time directly from secondary storage causes severe I/O bottlenecks. Lexical analyzers use an **Input Buffering** scheme with block reads:

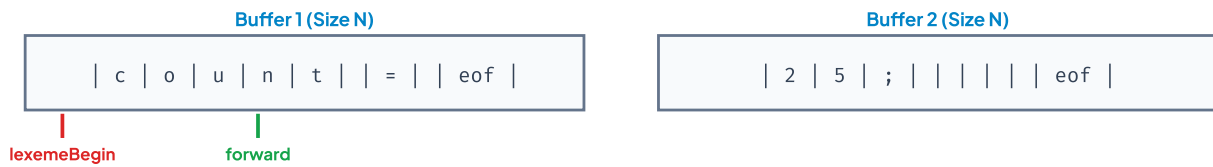


Figure 1.2: Two-Buffer Scheme with lexemeBegin and forward pointers & eof Sentinels

`lexemeBegin` Pointer: Marks the start of the current lexeme being recognized.

`forward` Pointer: Scans ahead one character at a time until a token boundary is identified.

Sentinels (`eof`): Appending an `eof` character at the end of each buffer reduces two boundary tests per character (end of buffer check + EOF check) down to a single test.

Topic 5 & 6: Specification & Recognition of Tokens

Regular Expression Operators & Precedence

1. **Kleene Star (*)**: Zero or more repetitions (Highest Precedence).
2. **Concatenation**: Sequence of symbols (Medium Precedence).
3. **Alternation (|)**: Logical OR / choice (Lowest Precedence).

Topic 7: DFA Directly from Regular Expressions (Direct Method)

The Direct Method constructs a minimal DFA directly from an augmented regular expression $(r)\#$ without generating an intermediate Thompson NFA.

The 4 Fundamental Functions: `nullable`, `firstpos`, `lastpos`, `followpos`

Node Type n	`nullable(n)`	`firstpos(n)`	`lastpos(n)`
Leaf ϵ	true	$\emptyset$	$\emptyset$
Leaf Position i	false	$\{i\}$	$\{i\}$
OR: $c_1 \mid c_2$	`nullable(c1)    nullable(c2)`	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
Cat: $c_1 \cdot c_2$	`nullable(c1) && nullable(c2)`	if nullable(c_1) then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else $\text{firstpos}(c_1)$	if nullable(c_2) then $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ else $\text{lastpos}(c_2)$
Star: c_1^*	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

`followpos(i)` Invariant Rules:

1. For Cat node $n = c_1 \cdot c_2$: For every position $i \in \text{lastpos}(c_1)$, $\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(c_2)$.
2. For Star node $n = c_1^*$: For every position $i \in \text{lastpos}(c_1)$, $\text{followpos}(i) = \text{followpos}(i) \cup \text{firstpos}(c_1)$.

Step-by-Step Solved Problem: Direct DFA Construction for $(a \mid b)^*abb\#$

Step 1: Assign Position Indices: $(a_1 \mid b_2)^* \cdot a_3 \cdot b_4 \cdot b_5 \cdot \#_6$

Step 2: Compute `followpos` Table:

Node Position i	Symbol	`followpos(i)`
1	a	$\{1, 2, 3\}$
2	b	$\{1, 2, 3\}$
3	a	$\{4\}$
4	b	$\{5\}$
5	b	$\{6\}$
6	$\#$	$\emptyset$

Step 3: DFA States & Transitions:

- $S_0 = \text{firstpos}(\text{root}) = \{1, 2, 3\}$
- $\delta(S_0, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_1$
- $\delta(S_0, b) = \text{followpos}(2) = \{1, 2, 3\} = S_0$
- $\delta(S_1, a) = \text{followpos}(1) \cup \text{followpos}(3) = \{1, 2, 3, 4\} = S_1$
- $\delta(S_1, b) = \text{followpos}(2) \cup \text{followpos}(4) = \{1, 2, 3, 5\} = S_2$
- $\delta(S_2, b) = \text{followpos}(2) \cup \text{followpos}(5) = \{1, 2, 3, 6\} = S_3$ (Accepting, since $6 \in S_3$)

M1 Active Recall & Exam-Important Question Bank

Q1. Explain the 6 phases of a compiler with a neat translation diagram for the statement `position = initial + rate \* 60`. (10 Marks)

- 1. Lexical Analysis:** Produces token stream: $\langle \text{id}_1 \rangle \langle = \rangle \langle \text{id}_2 \rangle \langle + \rangle \langle \text{id}_3 \rangle \langle * \rangle \langle 60 \rangle$.
- 2. Syntax Analysis:** Builds parse tree showing operator precedence ($*$ over $+$).
- 3. Semantic Analysis:** Checks types and inserts `inttofloat(60)` conversion.
- 4. Intermediate Code Generation:** Emits Three-Address Code:
``t1 = inttofloat(60); t2 = id3 * t1; t3 = id2 + t2; id1 = t3``
- 5. Code Optimization:** Constant propagation & strength reduction: ``t1 = id3 * 60.0; id1 = id2 + t1``
- 6. Code Generation:** Emits target assembly: ``LDF R2, id3; MULF R2, #60.0; ADDF R2, id2; STF id1, R2``.

Q2. Explain the Two-Buffer scheme and the role of Sentinels in input buffering. (8 Marks)

- Two-Buffer Scheme:** Uses two contiguous blocks of size N in memory. While the scanner processes characters in Buffer 1 using ``lexemeBegin`` and ``forward`` pointers, Buffer 2 can be asynchronously filled via a single system I/O read call.
- Role of Sentinels:** Without sentinels, testing every character requires two comparisons: (1) ``if (forward == buffer_end)`` and (2) test character. Placing an ``eof`` sentinel at the end of each buffer reduces this to a single check: only when an ``eof`` character is encountered does the scanner check whether it has reached a buffer boundary or the genuine end of the source file.